

Running the Scream Task + Stop Signal Task Battery

Task Type	Multi-day
Last edited time	@July 2, 2025 10:23 AM

How to run tasks:

[Timing](#)

[Links](#)

[Code Changelog \(in order from most recent to least recent\)](#)

How to run tasks:

1. Clone kaley's Conte Tasks Github <https://github.com/kaleyjoss/conte-p3-tasks>
2. In your terminal, cd to that folder on your local machine
3. Run these lines:

```
export FLASK_APP=app.py
export FLASK_ENV=development # don't include this if not testing
export FLASK_RUN_PORT=9000 #change this depending on which port you have free
flask run --host=0.0.0.0 # This is if you want it to be accessible to others on your IP address or on whatever server you are using
```

It should return something like:

```
* Serving Flask app 'app.py'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:9000 #current machine, not portable
```

* Running on `http://YOUR_IP_ADDRESS:9000` #your IP address, portable to others

4. Then, for local testing, you can go to a browser and input:

`http://127.0.0.1:9000/?PROLIFIC_PID=testuser` #change 'testuser' to whatever ID

If you're not on a server, others can access this through

`http://YOUR_IP_ADDRESS:9000/?PROLIFIC_PID=testuser`

If you're on a server, change 'YOUR_IP_ADDRESS' to the server address

For example, on nivlab servers, they would access on here:

`https://nivlab-rldm-dev-07.princeton.edu/?PROLIFIC_PID=testuser`

Timing

Stop signal task:

- Using default settings: ~6mins, ~1min per block
 - average 50s per block my timing, 1min08s maximum (no key presses)
 - 64 trials per block, 4 blocks
- Can adjust to make it 6, 8 blocks → 8min, 10min

Risk Task

- ~12mins (whole task including instructions)
 - ~5min 30s per block, 2 blocks

Links

Github: <https://github.com/fredvbrug/STOP-IT/tree/master>

Gili Explains Nivturk

Nivturk Documentation: <https://nivlab.github.io/nivturk/>

[How to Serve NivTurk on Prolific](#)

[Stop signal lit review](#)

Code Changelog (in order from most recent to least recent)

KJ060425

diff nivturk-plugins.js: <https://www.diffchecker.com/IWU1CwUh/>

- on_success save/redirect functionality
 - fixed by putting copy into stop.html
- risk still works, just uncomment + change main.html

KJ060225

diff stop.html: <https://www.diffchecker.com/d8GzPsAn/>

diff __init__.py: <https://www.diffchecker.com/8RSYwNcb/>

- got save functionality to work effectively
 - added save_data_stopTask to __init__.py
 - made appendData call save_data_stopTask instead of save_data_php
- now need to get the workerID from prolific to populate
 - took out acqext in timeline for testing (from main.html)

- added if id == 'prolific' logic
- changed to prolific
- changed logic of id text parsing so that it wouldn't overwrite prolific
- changed initVarJs since that is js 7s

KJ053025

- added a way to link the stimulus images correctly
- found save data php doesn't work

KJ 052925

- added way to get workerId from other tasks into stop signal task {{workerId}}

KJ 052825

diff experiment.py: <https://www.diffchecker.com/Dd10PPd1/>

diff main.html <https://www.diffchecker.com/ijX51WWR/>

- Added risk task to `experiment.py` `@bp.route('/experiments/stop'`
 - backend— added serving adaptation
- Added `experiment.html` from stop task to `templates/experiments/stop.html`
- added `jspsych-6.0.5` to `static/lib`, changed script src in `stop.html` to the new path to js and jspsych package (replace `/js/` with `../../static/lib` for packages and `../../static/js/stop` for standalone (not within version folder) js files/plugins)
 - also put configuration .js files in `../../static/js/stop`

KJ 052725:

1. Conda environment with 3.9 (3.7 wasn't available)
2. read all documentation

3. Investigated setup

▼ investigation into subject id

WorkerID in NivTurk is set in [experiment.py](#)

```
## Record incoming metadata.
info = dict(
    workerId    = request.args.get('PROLIFIC_PID'), # Prolific metadata
    assignmentId = request.args.get('SESSION_ID'),  # Prolific metadata
    hitId       = request.args.get('STUDY_ID'),     # Prolific metadata
    subId       = gen_code(24),                     # NivTurk metadata
    address     = request.remote_addr,              # NivTurk metadata
    user_agent  = request.user_agent.string,        # User metadata
    code_success = cfg['PROLIFIC'].get('CODE_SUCCESS', gen_code(8).upper()),
    code_reject  = cfg['PROLIFIC'].get('CODE_REJECT', gen_code(8).upper()),
)
```

Which then defines the path of a log file

```
## Parse log file.
with open(os.path.join(session['metadata'], info['workerId']), 'r') as f:
    logs = f.read()

## Extract subject ID.
info['subId'] = re.search('subId\t(.*)\n', logs).group(1)
```

where it can then extract the subId

- from prolific?

Stop signal task

ID in SST is set by experiment.html, and added to the datafile

```
jsPsych.data.addProperties({
```

```
  // if enabled below, get participant's id from participant and add it to the
  datafile.
```

```
  // the prompt is declared in the configuration/text_variables.js file
```

```
  var participant_id = {
```

```
    type: 'survey-text',
```

```
    questions: [{
```

```
      prompt: subjID_instructions,
```

```
      required: true
```

```
    }, 1,
```

```
    button_label: label_next_button,
```

```
    on_finish: function(data) {
```

```
      var responses = JSON.parse(data.responses);
```

```
      var code = responses.Q0;
```

```
      jsPsych.data.addProperties({
```

```
        participantID: code
```

```
      });
```

```
    }
```

```
};
```

It's referenced by the app.R script by parsing the datafile

```
subj_idx <- data()$participantID
```

which assigns to 'server' the function `function` , which calls 'read.csv' on `input$csvs$datapath`

```
server <- function(input, output) {  
  data<-eventReactive(input$process,({  
    do.call("rbind", (lapply(input$csvs$datapath, function(i){  
      read.csv(i, header=TRUE, na.strings=c("null"), colClasses=c("correct"="factor"))}))  
    })))  
})
```

flask redirect—

- maybe could also send subject id?
- or find another flask variable which would allow sending a subject ID

```
return render_template('experiments/stop.html', workerId=session  
['workerId'], assignmentId=session['assignmentId'], hitId=session['hitId'],  
code_success=session['code_success'], code_reject=session['code_reject'])
```

- the workerId is being sent to the frontend

